CSE 102 Assignment 7 Report

Introduction

In this assignment, I implemented a terminal-based version of the classic Minesweeper game using the C programming language. The program randomly generates a game board of size N x N where $2 \leq N \leq 10$, places mines randomly, and allows the user to uncover cells or undo previous moves. The main features include:

- Random board generation
- Recursive flood fill when a zero-adjacent-mine cell is uncovered
- A stack structure for undoing moves
- Saving game moves to a file at the end

Game Board Generation

At the beginning of the program, the board size is determined randomly:

```
N = rand() % 9 + 2; // Random board size between 2 and 10
```

Each cell has a 1/5 probability of containing a mine. The mines are stored in a dynamically allocated 2D array. After generating the board, it is saved to a file called map.txt:

```
void generate_map() {
    srand(time(NULL));
    ...
    FILE *fp = fopen("map.txt", "w");
    ...
    fclose(fp);
}
```

Stack Implementation for Undo Feature

The undo feature is implemented using two arrays (move_rows and move_cols) and a move_top variable, simulating a stack behavior:

```
void push(int row, int col) {
    if (move_top < MAX_MOVES) {
        move_rows[move_top] = row;
        move_cols[move_top] = col;
        move_top++;
    }
}
```

When the user types "undo", the last move is removed:

```
void pop() {
    if (move_top > 0)
        move_top--;
}
```

The stack checks whether it is empty before allowing an undo:

```
int isEmpty() {
    return move_top == 0;
}
```

Gameplay Mechanics

The player can either:

- Enter a move by specifying row and column, or
- Type "undo" to reverse the last move.

If the selected cell contains a mine, the game ends. Otherwise, the number of adjacent mines is displayed. If the count is zero, a recursive flood fill reveals neighboring cells:

```
void flood_fill(int row, int col) {
    ...
    if (count == 0) {
        flood_fill(row - 1, col - 1);
        flood_fill(row - 1, col);
        flood_fill(row - 1, col + 1);
        flood_fill(row, col - 1);
        flood_fill(row, col + 1);
        flood_fill(row + 1, col - 1);
        flood_fill(row + 1, col);
        flood_fill(row + 1, col + 1);
    }
}
```

The player's view is updated in a separate 2D array, displaying:

- # for hidden cells
- 0-8 for revealed cells
- * for a mine

The board is reprinted after every move **void print_board();**

Win/Loss Conditions

- The player wins if all non-mine cells are revealed.
- The player loses immediately after uncovering a mine.

The win condition is checked using **int check_win();**

At the end of the game, a message is printed based on the result:

- "Congratulations! You won!"
- "BOOM! You hit a mine. Game Over."

Saving the Move History

After the game ends, the sequence of all moves is saved to a file moves.txt:

```
void save_moves() {
    FILE *fp = fopen("moves.txt", "w");
    fprintf(fp, "--- Game Moves ---\n");
    ...
    fclose(fp);
}
```

The file includes a formatted list of all moves made by the player and the total number of moves.

Outputs

```
   0 1 2 3 4 5 6 7
0  # # # # # # # #
1  # # # # # # # #
2  # # # # # # # #
3  # # # # # # # #
4  # # # # # # # #
5  # # # # # # # #
6  # # # # # # # #
7  # # # # # # # #

Enter move (row col) or 'undo': 1 5
```

```
   0 1 2 3 4 5 6 7
0  # # # # # # # #
1  # # # # # 0 # #
2  # # # # # # # #
3  # # # # # # # #
4  # # # # # # # #
5  # # # # # # # #
6  # # # # # # # #
7  # # # # # # # #

Enter move (row col) or 'undo': 3 6
```

```
   0 1 2 3 4 5 6 7
0  # # # # # # # #
1  # # # # # 0 # #
2  # # # # # # # #
3  # # # # # # 1 #
4  # # # # # # # #
5  # # # # # # # #
6  # # # # # # # #
7  # # # # # # # #

Enter move (row col) or 'undo': 2 7
```

```
   0 1 2 3 4 5 6 7
0  # # # # # # # #
1  # # # # # 0 # #
2  # # # # # # # 1
3  # # # # # # 1 #
4  # # # # # # # #
5  # # # # # # # #
6  # # # # # # # #
7  # # # # # # # #

Enter move (row col) or 'undo': 2 6
```

```
   0 1 2 3 4 5 6 7
0  # # # # # # # #
1  # # # # # 0 # #
2  # # # # # # 1 1
3  # # # # # # 1 #
4  # # # # # # # #
5  # # # # # # # #
6  # # # # # # # #
7  # # # # # # # #

Enter move (row col) or 'undo': undo
```

```
    0 1 2 3 4 5 6 7
 0  # # # # # # # #
 1  # # # # # 0 # #
 2  # # # # # # # 1
 3  # # # # # # 1 #
 4  # # # # # # # #
 5  # # # # # # # #
 6  # # # # # # # #
 7  # # # # # # # #

Enter move (row col) or 'undo': 3 7

    0 1 2 3 4 5 6 7
 0  # # # # # # # #
 1  # # # # # 0 # #
 2  # # # # # # # 1
 3  # # # # # # 1 *
 4  # # # # # # # #
 5  # # # # # # # #
 6  # # # # # # # #
 7  # # # # # # # #

BOOM! You hit a mine. Game Over.


...Program finished with exit code 0
Press ENTER to exit console.
```

main.c | map.txt | moves.txt

```
1  * . . . . . . .
2  . . . . . . . .
3  * . * . . . . .
4  . . * . . . . *
5  . . . * . . . .
6  * . . . . . . .
7  . . . . . . . .
8  . * * . . . . .
9
```

main.c | map.txt | moves.txt

```
1  --- Game Moves ---
2  Move 1: (Row 1, Col 5)
3  Move 2: (Row 3, Col 6)
4  Move 3: (Row 2, Col 7)
5  Move 4: (Row 3, Col 7)
6  Total Moves: 4
7
```

Conclusion

This project demonstrated the use of arrays, dynamic memory allocation, file I/O, recursive functions, and a manually implemented stack data structure in C programming. It also improved skills in handling user input and managing game state for a terminal-based application.

Some important points while implementing:

- Careful dynamic memory management (malloc/free).
- Recursive flood fill implementation without infinite loops.
- Proper use of randomization for board and mines generation.
- Correct undo behavior using a custom stack without structs.